

맨] 범그 구sc 그세일의사참추t 좀쳐악 살즈구색참알se 인쳐좀알 m제쳐중f 열i 색추세 계사좀쳐추
계사열악창즈그사참악

q과격가당겸	969<	2	-계개만가개	Mc c		4
T구공공체	%	1	%		4	
Mc c	-Wt hn 2YrU.			2		
4	hTgU			4		
%		-h가만가개공계갈%	2	- D	.	2
		4				2
	4					
D		-86)	.	9386	4	
	DhTgU			4		
	4T구공공체		hTgU			4

]]:F m계계

D

8DU가계갈공가게갈

가격 필터: price ∈ [100, 500]
선택도: 10% (1,000,000개 중 100,000개)

단계:
1. 범위 스캔으로 100,000개 항목의 벡터ID 획득
2. 100,000개 벡터에서만 ANN 검색

문제: 범위 필터 선택도가 높으면 느림
선택도 80% → 800,000개 벡터 검색 필요

9Dh가계갈공가게갈

단계:
1. 모든 벡터에서 top-K 유사 벡터 검색
2. 검색 결과에서 가격 필터 적용

문제: 필터 선택도가 낮으면, top-K 결과에 필터 조건을 만족하는 벡터가 거의 없을 수 있음
→ K를 크게 증가시켜야 함 → 검색 비용 급증

: D

가격에 대한 B-Tree 인덱스 유지
벡터 HNSW 인덱스 유지

검색 시:
1. B-Tree로 가격 범위의 ID 얻기
2. HNSW에서 해당 ID 벡터들만 방문

문제: 두 인덱스 동기화 필수
메모리 오버헤드 2배
탐색 중 인덱스 간 점프로 캐시 미스 증가

]]:G 42사일의사점추 증차15

D

가격 범위로 세그먼트 분할:

Segment 0: [0, 100) → 10,000 벡터
Segment 1: [100, 200) → 15,000 벡터
Segment 2: [200, 300) → 12,000 벡터
Segment 3: [300, 500) → 20,000 벡터
Segment 4: [500, ∞) → 40,000 벡터

각 세그먼트 내 HNSW 그래프:

Seg0: [벡터 구조] -- 연결 --
Seg1: [벡터 구조] -- 연결 --
Seg2: [벡터 구조] -- 연결 --
...

세그먼트 간 크로스링크:

Seg0 ↔ Seg1 ↔ Seg2 ...
(최근접 세그먼트 간의 경계 벡터들이 연결)

D

D결계가야 r] 2l t

Dl y공인력문G,가림 공화개교

Dey공이 r공계공개

D결계가F =66. Mc S -ly공인력문G,가림 공화개교

]]:N

D

```
def build_segment_graph(vectors, metadata, partition_attr):  
    # 1단계: 메타데이터 기준으로 세그먼트 분할  
    segments = partition_by_attribute(vectors, metadata, partition_attr)  
  
    # 2단계: 각 세그먼트 내에서 HNSW 구축  
    segment_graphs = {}  
    for seg_id, vectors_in_seg in segments.items():  
        segment_graphs[seg_id] = build_hnsw(vectors_in_seg)  
  
    # 3단계: 세그먼트 간 크로스링크 구축  
    for seg_i, seg_j in adjacent_segments(segments):  
        create_cross_segment_edges(  
            segment_graphs[seg_i],  
            segment_graphs[seg_j],  
            num_edges=10 # 세그먼트당 10개 엣지  
        )  
  
    return segment_graphs, segment_boundaries
```

D

세그먼트 크기 s:

- 너무 작음 ($s < 1000$): 세그먼트 수 증가 → 메모리 오버헤드
- 너무 큼 ($s > 100000$): 한 세그먼트에 대부분 벡터 → 필터링 효과 감소
- 최적: $s \approx \sqrt{n}$ (n = 전체 벡터 수)

예: 1,000,000개 벡터 → 세그먼트당 ~1,000개
→ 약 1,000개 세그먼트

]]:R

D

```
def range_filtered_search(query_vector, range_filter, top_k):  
    # 1단계: 필터 조건에 맞는 세그먼트 식별  
    target_segments = identify_segments(range_filter)  
    # 예: price ∈ [100, 500]이면 Segment 1, 2, 3 선택
```

```
# 2단계: 각 세그먼트에서 독립적으로 ANN 검색
segment_candidates = {}
for seg_id in target_segments:
    segment_candidates[seg_id] = hnsf_search(
        segment_graphs[seg_id],
        query_vector,
        k=top_k * len(target_segments) # 각 세그먼트에서 충분히 가져오기
    )

# 3단계: 모든 세그먼트 결과를 병합 및 정렬
all_candidates = merge_sorted(segment_candidates.values())

# 4단계: 상위 top_k 반환
return all_candidates[:top_k]
```

D

쿼리: price ∈ [100, 500]

필터링 없음: 1,000,000개 벡터 모두 검색 필요

세그먼트 그래프:

- Segment 1: [100, 200) → 0
- Segment 2: [200, 300) → 0
- Segment 3: [300, 500) → 0
- 나머지 세그먼트는 스킵

결과: 47,000개 벡터만 검색

성능 개선: 1,000,000 / 47,000 ≈ 21배

]]:S

D

Segment 1: [100, 200)

Segment 2: [200, 300)

벡터 A: 벡터값 = [0.5, 0.3, ...], 가격 = 199 → Seg 1

벡터 B: 벡터값 = [0.5, 0.31, ...], 가격 = 200 → Seg 2

두 벡터는 벡터 공간에서 매우 가깝지만 다른 세그먼트에 속함!

→ 크로스 세그먼트 엣지가 해결

D

Segment 1의 경계 벡터들이 Segment 2의 경계 벡터들과 연결되어 있으므로,

Segment 1에서 탐색 시 필요하면 Segment 2로 "넘어갈" 수 있음

하지만: 필터 조건 밖인 세그먼트로는 가면 안 됨!

→ 크로스 엣지는 필터 세그먼트 내에서만 따라가야 함

]]:[

D

price ∈ [100, 500]

→ Segment 1, 2, 3 선택 (직관적)

D

price ∈ [100, 500] AND rating ∈ [4, 5]

문제: 세그먼트를 price로만 분할하면 rating 조건을 활용 못함

해결책 1: 계층 세그먼트화

└ 1단계: price로 분할

| 각 세그먼트를 다시 rating으로 분할

| → 다차원 세그먼트 격자(grid)

└ 비용: 메모리 증가 (세그먼트 수 = n_price_segs × n_rating_segs)

- └ 메인 세그먼트: price 기준
- └ 검색 중: rating 조건도 동시 확인
- └ 메모리 효율적
- └ 계산 비용: 추가 필터 검사

D
DMY 그래프-8: 6a 결과와 공식 2c p r y 그래프 8 개가 각각

	U h	hi U	hTgU	
L3333333L3333L3333L33333L33333L				
8)	96	: 66	99	hi U 8:
=)	B6	9=6	C6	hi U 9B
86)	8=6	996	8?6	hi U 8A
=6)	?66	?66	?86	
C6)	8=66	=66	=86	Ui h 9C

크로스 세그먼트 엣지: 약 50 MB
세그먼트 경계 정보: 약 10 MB

hTgU

- 정확한 선택도:
- $\sigma = 15\%$ → SERF 사용 → 빠름 (150ms)

부정확한 선택도:

- 추정값 50% → FTS 선택 → 느림 (600ms)
- 추정값 10% → SERF 선택하지만 실제 선택도 50% → 느림 (610ms)

T기형염색체	hTgU	D
--------	------	---

Exqutor의 경량 선택도 모델 오류:

- $\pm 10\%$ 범위: 최적 전략 선택 가능
- $\pm 30\%$ 범위: 1.3배 성능 저하
- $\pm 50\%$ 범위: 2배 이상 성능 저하

T구형염색체 hTgU D

T구멍크기	hTgU	4
-------	------	---

범위 필터 조건 입력
↓
선택도 추정 모듈 (논문 [70] 경량 ML 모델)
↓
선택도: $\sigma = 0.12$ (12%)
↓
쿼리 유틸마이저

└ $\sigma < 30\%$ → SERF 세그먼트 검색 선택

└ $\sigma \geq 30\%$ → 전체 HNSW 검색 후 필터

↓

실행 및 성능 측정

84	D	관고	2	I	
94	D		2	I	
: 4	D	결계가 Mc S 계공개안 Mc S y공단역개			I
<4	D		- D		2 .2Wt hn
		I			
=4	D			I	
?4	D		2	I	